

Claude Code Agent Loop 架构深度解析

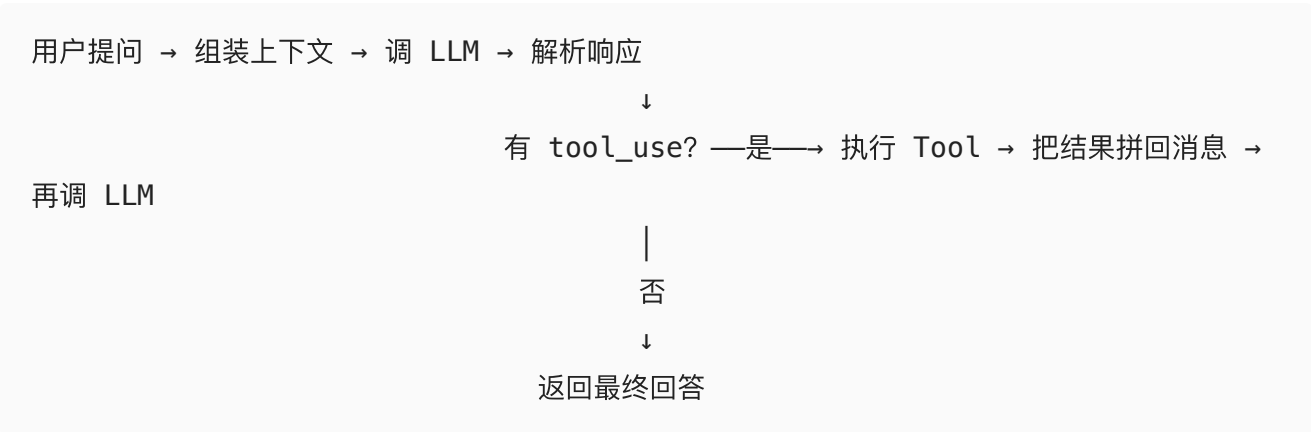
Author: **Daniel Walnut**, Coauthor of [DeerFlow](#)

📖 摘要

本文基于从 source maps 恢复的 Claude Code 源码进行逆向分析，完整揭示其 **Agent Loop**、**Tool** 系统、**Skill** 系统、**Memory/Context** 管理以及 **Multi-Agent** 协作架构的设计原理。面向希望理解生产级 AI Agent 系统内部工作方式的工程师。

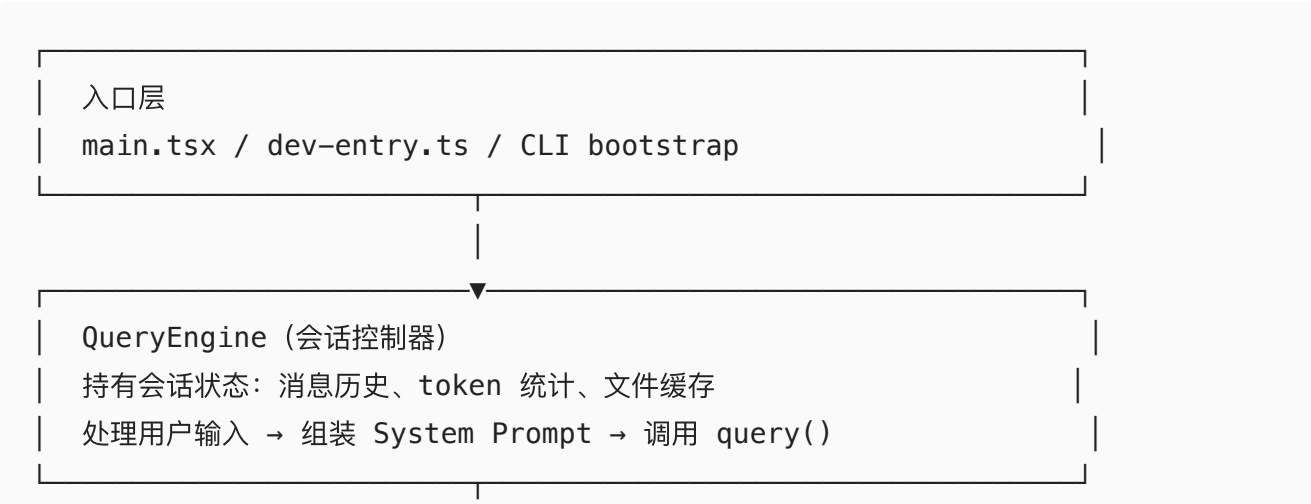
一、全局架构鸟瞰

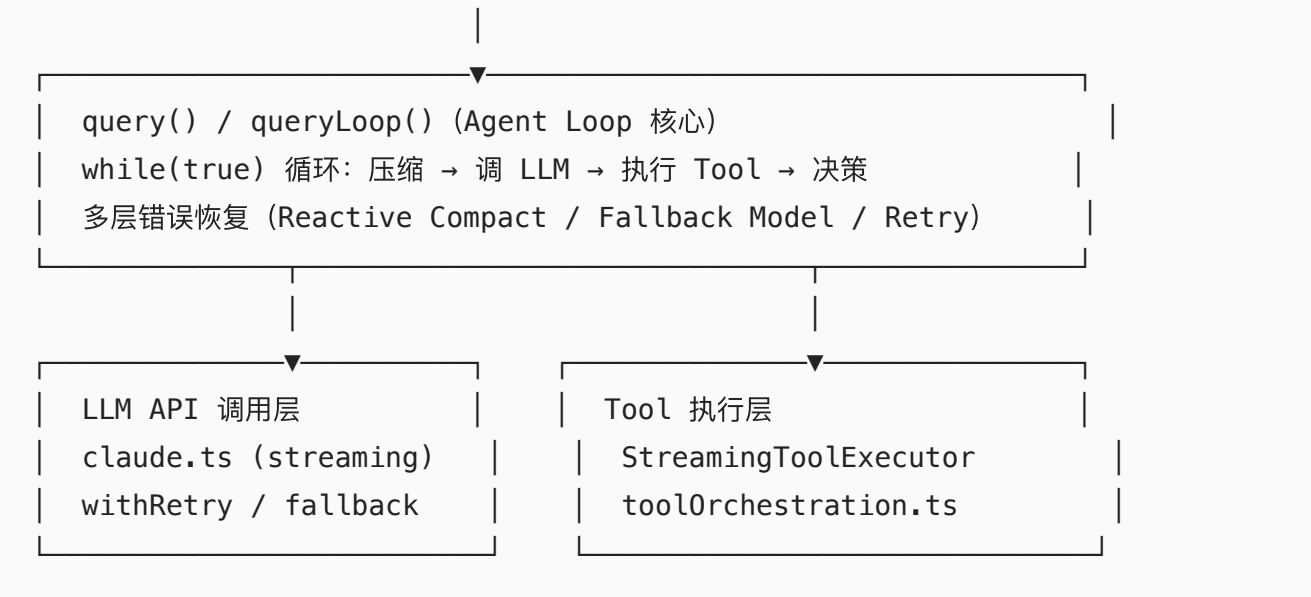
Claude Code 的核心是一个 **工具增强的对话式 Agent 系统**。最简化的心智模型：



这个“调 LLM → 执行 Tool → 再调 LLM”的循环，就是 **Agent Loop**。Claude Code 的工程复杂度在于围绕这个简单循环构建了一整套生产级基础设施：上下文压缩、权限控制、错误恢复、多 Agent 协同、持久化记忆等。

核心模块关系





关键源文件索引

文件	职责
src/QueryEngine.ts	会话控制器，每 conversation 一个实例
src/query.ts	Agent Loop 核心循环（queryLoop）
src/Tool.ts	Tool 接口定义和类型系统
src/tools.ts	Tool 注册表（所有内置工具的单一来源）
src/context.ts	用户上下文和系统上下文的加载
src/constants/prompts.ts	System Prompt 的组装逻辑
src/memdir/memdir.ts	持久化记忆系统（Auto Memory）
src/skills/loadSkillsDir.ts	Skill 加载和管理
src/tools/AgentTool/runAgent.ts	子 Agent 的创建和运行
src/Task.ts	后台任务的基础抽象

二、Agent Loop：核心循环

QueryEngine：会话的入口

每个对话由一个 QueryEngine 实例管理。它的 submitMessage() 方法是整个系统的入口点：

```
// src/QueryEngine.ts（简化）
class QueryEngine {
  private mutableMessages: Message[] // 对话历史
  private totalUsage: NonNullableUsage // token 统计
```

```

private readFileState: FileStateCache // 文件内容缓存

async *submitMessage(prompt): AsyncGenerator<SDKMessage> {
  // 1. 组装 System Prompt
  const systemPrompt = [defaultSystemPrompt, memoryPrompt, appendPrompt]

  // 2. 处理用户输入 (解析 /slash 命令等)
  const { messages, shouldQuery } = await processUserInput(prompt)

  // 3. 如果不需要调 LLM (如纯 /slash 命令), 直接返回
  if (!shouldQuery) { yield result; return }

  // 4. 进入核心 Agent Loop
  for await (const message of query({ messages, systemPrompt, ... })) {
    // 5. 处理各种消息类型
    yield normalizedMessage
  }

  // 6. 返回最终结果
  yield { type: 'result', total_cost_usd, usage, ... }
}

```

🔗 关键设计

整个链路使用 **AsyncGenerator + yield**，实现了从 LLM streaming 到 Tool 执行到最终结果的全流式管道。用户可以实时看到 LLM 的输出，不需要等待整个 turn 完成。

queryLoop: Agent 心跳

queryLoop() 是真正的 Agent Loop——一个 while(true) 无限循环。每次迭代是一个"思考 → 行动 → 观察"周期：

```

// src/query.ts (简化)
async function* queryLoop(params) {
  let state: State = { messages, turnCount: 1, ... }

  while (true) {
    // ===== Phase 1: 上下文准备 =====
    let messagesForQuery = getMessagesAfterCompactBoundary(messages)
    messagesForQuery = await applyToolResultBudget(messagesForQuery) //
    大结果存磁盘
  }
}

```

```

    messagesForQuery = snipCompactIfNeeded(messagesForQuery) //
裁剪旧消息
    messagesForQuery = await microcompact(messagesForQuery) //
细粒度压缩
    messagesForQuery = await autocompact(messagesForQuery) //
超阈值则 LLM 总结

// ===== Phase 2: 调用 LLM =====
for await (const message of callModel({ messages, systemPrompt, tools
})) {
    yield message // 流式推送给上层
    if (hasToolUseBlocks(message)) {
        needsFollowUp = true
        streamingToolExecutor.addTool(toolBlock, message) // 边流边执行
    }
}

// ===== Phase 3: 决策 =====
if (!needsFollowUp) {
    yield* handleStopHooks(...)
    return { reason: 'completed' }
}

// ===== Phase 4: 执行剩余 Tools =====
for await (const update of toolExecutor.getRemainingResults()) {
    yield update.message
    toolResults.push(update.message)
}

// ===== Phase 5: 准备下一轮 =====
state = {
    messages: [...messagesForQuery, ...assistantMessages,
...toolResults],
    turnCount: turnCount + 1,
}
continue // → 回到 Phase 1
}
}

```

消息类型体系

Agent Loop 中流转的消息类型 (src/types/message.ts):

类型	说明	示例
user	用户消息（含 tool_result）	用户输入、Tool 执行结果
assistant	LLM 响应	文本回复、tool_use 调用
system	系统消息	API 错误、compact boundary、警告
progress	进度更新	Tool 执行进度、Agent 进度
attachment	附件消息	记忆文件、structured output
stream_event	流事件	message_start / message_delta / message_stop
tombstone	删除标记	标记需从 UI 中移除的消息
tool_use_summary	摘要	压缩多个 tool 调用的简要描述

错误恢复：Agent 的韧性

🔗 生产级核心

这是 Claude Code 与 demo 级 Agent 的关键区别——LLM 调用不是总能成功的。

queryLoop 实现了多层错误恢复：

```
错误发生
├─ prompt_too_long (413)
│   └─ 1. Context Collapse drain (释放已暂存的折叠)
│       └─ 2. Reactive Compact (紧急用 LLM 压缩对话)
│           └─ 3. 都失败 → 报错退出
├─ max_output_tokens
│   └─ 1. 升级到 64K tokens 重试 (一次性)
│       └─ 2. 注入 "Resume directly" 恢复消息 (最多 3 次)
│           └─ 3. 恢复耗尽 → 显示原始错误
├─ model_fallback
│   └─ 切换到 fallback model 重试
├─ media_size_error
│   └─ Reactive Compact 移除大图后重试
└─ streaming 中止 (用户取消)
    └─ 为所有未完成的 tool_use 生成 synthetic tool_result
```

② 为什么需要 synthetic tool_result?

Anthropic API 要求每个 `tool_use` block 必须有对应的 `tool_result`。如果 streaming 被中断，已发出的 `tool_use` 没有 result，下一轮 API 调用会报错。所以系统必须生成占位的错误结果。

三、Tool 系统：能力边界

Tool 是 Claude Code 与外部世界交互的**唯一途径**。LLM 不能直接读文件、执行命令——只能通过 Tool 接口请求这些操作。

Tool 接口设计

每个 Tool 遵循统一接口（`src/Tool.ts`），核心方法分为四组：

```
type Tool<Input, Output, Progress> = {
  // ===== 身份 =====
  name: string // 工具名
  aliases?: string[] // 别名（向后兼容重命名）
  inputSchema: ZodType // 参数 schema (Zod 定义)
  searchHint?: string // ToolSearch 关键词提示

  // ===== 执行 =====
  call(args, context, canUseTool, parentMessage, onProgress)
    : Promise<ToolResult<Output>>

  // ===== 权限 =====
  checkPermissions(input, context): Promise<PermissionResult>
  validateInput?(input, context): Promise<ValidationResult>
  isReadOnly(input): boolean
  isDestructive?(input): boolean
  isConcurrencySafe(input): boolean

  // ===== 展示 =====
  prompt(options): Promise<string> // 生成 system prompt 描述
  renderToolUseMessage(input) // React 组件：渲染调用
  renderToolResultMessage?(output) // React 组件：渲染结果

  // ===== 高级 =====
  maxResultSizeChars: number // 超此大小，结果存磁盘
  shouldDefer?: boolean // 延迟加载（需先 ToolSearch）
}
```

```
interruptBehavior?(): 'cancel' | 'block'  
}
```

buildTool() 工厂函数

所有 60+ 个工具都通过 `buildTool()` 构建，提供 **Fail-Closed** 安全默认值——忘记声明 `isReadOnly`，默认 `false`（假设会写入），权限系统会要求用户确认。

```
const TOOL_DEFAULTS = {  
  isConcurrencySafe: () => false, // 默认不可并发  
  isReadOnly: () => false, // 默认假设会写  
  isDestructive: () => false,  
  checkPermissions: () => ({ behavior: 'allow' }), // 交给全局权限系统  
}
```

Tool 注册总览

`src/tools.ts` 中的 `getAllBaseTools()` 是所有内置工具的注册表：

核心工具（始终可用）：

- `BashTool` — 执行 shell 命令
- `FileReadTool` / `FileEditTool` / `FileWriteTool` — 文件操作
- `GlobTool` / `GrepTool` — 文件搜索（基于 ripgrep）
- `AgentTool` — 生成子 Agent
- `SkillTool` — 执行 Skill
- `WebFetchTool` / `WebSearchTool` — 网络操作
- `AskUserQuestionTool` — 向用户提问
- `EnterPlanModeTool` / `ExitPlanModeTool` — 计划模式

任务管理工具（feature-gated）：

- `TaskCreate` / `TaskGet` / `TaskUpdate` / `TaskList` — 任务 CRUD
- `SendMessageTool` — Agent 间消息
- `TeamCreateTool` / `TeamDeleteTool` — 团队管理

高级工具（环境/feature flag 决定）：

- `SleepTool`、`CronTools`、`MonitorTool`、`WorkflowTool`、`EnterWorktreeTool`、`ToolSearchTool` 等

工具池最终组装：内置工具 + MCP 工具 → 按名字排序（保证 prompt cache 稳定性）→ 去重（内置优先）

```
function assembleToolPool(permissionContext, mcpTools): Tools {
  const builtInTools = getTools(permissionContext)
  const allowedMcpTools = filterToolsByDenyRules(mcpTools,
permissionContext)
  return uniqBy(
    [...builtInTools].sort(byName).concat(allowedMcpTools.sort(byName)),
    'name'
  )
}
```

Tool 执行编排

Tool 执行有两种模式：

传统模式（toolOrchestration.ts）——按分区执行：

```
tool_use blocks
└─ partitionToolCalls()
   └─ [Grep, Glob, Read] → isConcurrencySafe=true → 并发执行
      └─ [FileEdit] → isConcurrencySafe=false → 串行执行
         └─ [Grep, Grep] → isConcurrencySafe=true → 并发执行
```

流式模式 (StreamingToolExecutor) ——性能优化亮点:

时间线 →→→→→→→→→→→→→→→→→→→→→→→

LLM Streaming: [text] [tool_use:A] [text] [tool_use:B] [tool_use:C] [end]

↓ ↓ ↓

Tool 执行: [开始执行A] [开始执行B] [开始执行C]

↓ ↓ ↓

[A完成] [B完成] [C完成]

🔄 边流边执行

LLM 还在输出后续内容时，已解析完的 tool 调用就开始执行了，显著降低端到端延迟。
Bash 出错时通过 `siblingAbortController` 取消同级进程。

LLM 还在输出后续内容时，已解析完的 tool 调用就开始执行了，显著降低端到端延迟。Bash 出错时通过 `siblingAbortController` 取消同级进程。

三层权限模型

第 1 层: Deny Rules 过滤 (工具注册时)

- └ 整个 tool 被禁用

第 2 层: Tool 自身 checkPermissions()
└ 工具级特殊逻辑 (如 BashTool 检查命令白名单)

第 3 层: 全局 canUseTool 回调
└ Hook 系统 (PreToolUse hooks)
└ Auto-mode 分类器 (判断操作风险)
└ UI 权限弹窗 (需用户确认)

PermissionMode: default (逐一确认) | auto (分类器判断) | plan (只读) |
bypassPermissions (全部允许)

四、Skill 系统：可扩展知识

Skill 是 Claude Code 的"外挂知识库"——允许在不修改核心代码的情况下扩展 Claude 的能力。

Skill 的本质

一个 Skill = **Markdown 文件 + YAML frontmatter**, 存放于
.claude/skills/<name>/SKILL.md :

```
---  
description: "Deploy applications to Vercel"  
when_to_use: "Use when the user requests deployment actions"  
allowed-tools: [Bash, FileEdit, FileRead]  
model: sonnet  
paths: ["src/deploy/**"]  
user-invocable: true  
context: fork  
---
```

Vercel Deployment Skill

Follow these steps to deploy:

1. Check if `vercel.json` exists using Read tool
2. Run `vercel deploy` using Bash tool
3. Return the deployment URL

Current skill directory: \${CLAUDE_SKILL_DIR}

两种触发路径

路径 1：用户主动调用 (/skill-name args)

```
用户输入 /deploy my-app
↓ processUserInput() 识别为 slash command
↓ SkillTool.call()
↓ skill.getPromptForCommand(args)
↓ 模板变量替换 + shell 命令执行
↓ 生成的 prompt 注入当前对话
↓ LLM 按 skill 指令执行
```

路径 2：LLM 自主调用

```
LLM 看到 system prompt 中的 skill 描述 (when_to_use 字段)
↓ 判断当前任务匹配某个 skill
↓ 调用 SkillTool(skill: "deploy", args: "my-app")
↓ 同上流程
```

7 层加载优先级

Skill 从多个来源加载，按优先级从高到低：

1. **policySettings** — 组织级策略 (~/.claude/managed/.claude/skills/)
2. **userSettings** — 用户个人 (~/.claude/skills/)
3. **projectSettings** — 项目级 (.claude/skills/ , 向上遍历到 \$HOME)
4. **--add-dir** — 命令行指定的额外目录
5. **legacy commands** — 旧版 .claude/commands/ (向后兼容)
6. **bundled** — 内置 skill (如 claude-api, verify)
7. **MCP skills** — 从 MCP server 动态加载

去重机制

通过 `realpath()` 解析符号链接，确保同一文件不被加载多次（即使通过不同路径访问）。

条件 Skill 和动态发现

条件 Skill：通过 `paths frontmatter` 声明，只有操作匹配路径时才激活：

```
paths: ["src/components/**/*.tsx"]
```

动态发现：操作子目录文件时，自动扫描沿途 `.claude/skills/`：

```
编辑 src/packages/auth/login.ts
  ↓ discoverSkillDirsForPaths()
  ↓ 检查 src/packages/auth/.claude/skills/
  ↓ 检查 src/packages/.claude/skills/
  ↓ 检查 src/.claude/skills/
  ↓ 发现新 skill → 注入上下文
```

五、Memory 与 Context 管理

⚠ 最复杂的子系统

如何在有限的上下文窗口中让 Agent 记住尽可能多的信息——这是 Claude Code 最具工程含量的部分。

System Prompt 组装

System Prompt 由多个分区组成 (`src/constants/prompts.ts`), 分为可缓存的静态区和每轮可能变化的动态区:

Static Sections (跨用户共享缓存)
├─ Intro: "You are an interactive agent..."
├─ System: 权限模式、hooks 说明
├─ Doing Tasks: 编码规范、提交规范
├─ Actions: 风险操作需确认
├─ Using Your Tools: 工具使用指南
├─ Tone & Style: 输出风格
└─ Output Efficiency: 简洁高效
┌─ == DYNAMIC BOUNDARY (Cache breakpoint) ==
Dynamic Sections (systemPromptSection 管理)
├─ session_guidance: 会话特定指导
├─ memory: Auto Memory 指令
├─ env_info: OS/CWD/Shell 信息
├─ language: 语言偏好
├─ output_style: 自定义输出风格
├─ mcp_instructions: MCP 服务器指令
├─ scratchpad: 共享工作区说明
└─ token_budget: Token 预算控制

🔗 Prompt Cache 优化

`static sections` 跨用户共享，是 API 的"缓存前缀"。`systemPromptSection()` 函数只在 `/clear` 或 `/compact` 时重新计算。`DANGEROUS_uncachedSystemPromptSection` 每轮重算（如 MCP 指令——server 可能随时连断）。

用户上下文

每次 API 调用 prepend 两段上下文（`src/context.ts`）：

```
// userContext - prepend 到用户消息前
const userContext = {
  claudeMd: "...", // CLAUDE.md 文件内容（项目指令）
  currentDate: "Today's date is 2026-04-03.",
}

// systemContext - append 到 system prompt 后
const systemContext = {
  gitStatus: "Branch: main\nStatus: (clean)\nRecent commits: ...",
}
```

CLAUDE.md 从多位置收集并合并：

- `~/.claude/CLAUDE.md`（全局）
- `<project>/.claude/CLAUDE.md`（项目级）
- 向上遍历目录直到 `$HOME`

Auto Memory：持久化记忆

`src/memdir/memdir.ts` 实现了基于文件的持久化记忆系统：

```
~/.claude/projects/<project-slug>/memory/
├─ MEMORY.md           ← 索引文件（最多 200 行 / 25KB）
├─ user_preferences.md ← 用户偏好
├─ project_arch.md     ← 项目架构笔记
├─ feedback_testing.md ← 测试反馈
├─ logs/               ← KAIROS 模式的日志
└─ 2026/04/2026-04-03.md
```

工作原理：

1. Memory 指令通过 `loadMemoryPrompt()` 注入 system prompt
2. LLM 看到"你有一个持久化记忆系统在 `<path>`"

3. LLM 通过 `FileWrite` tool 直接写入记忆文件
4. 下次会话启动时，`MEMORY.md` 内容加载到 `userContext`

4 类记忆分类法：

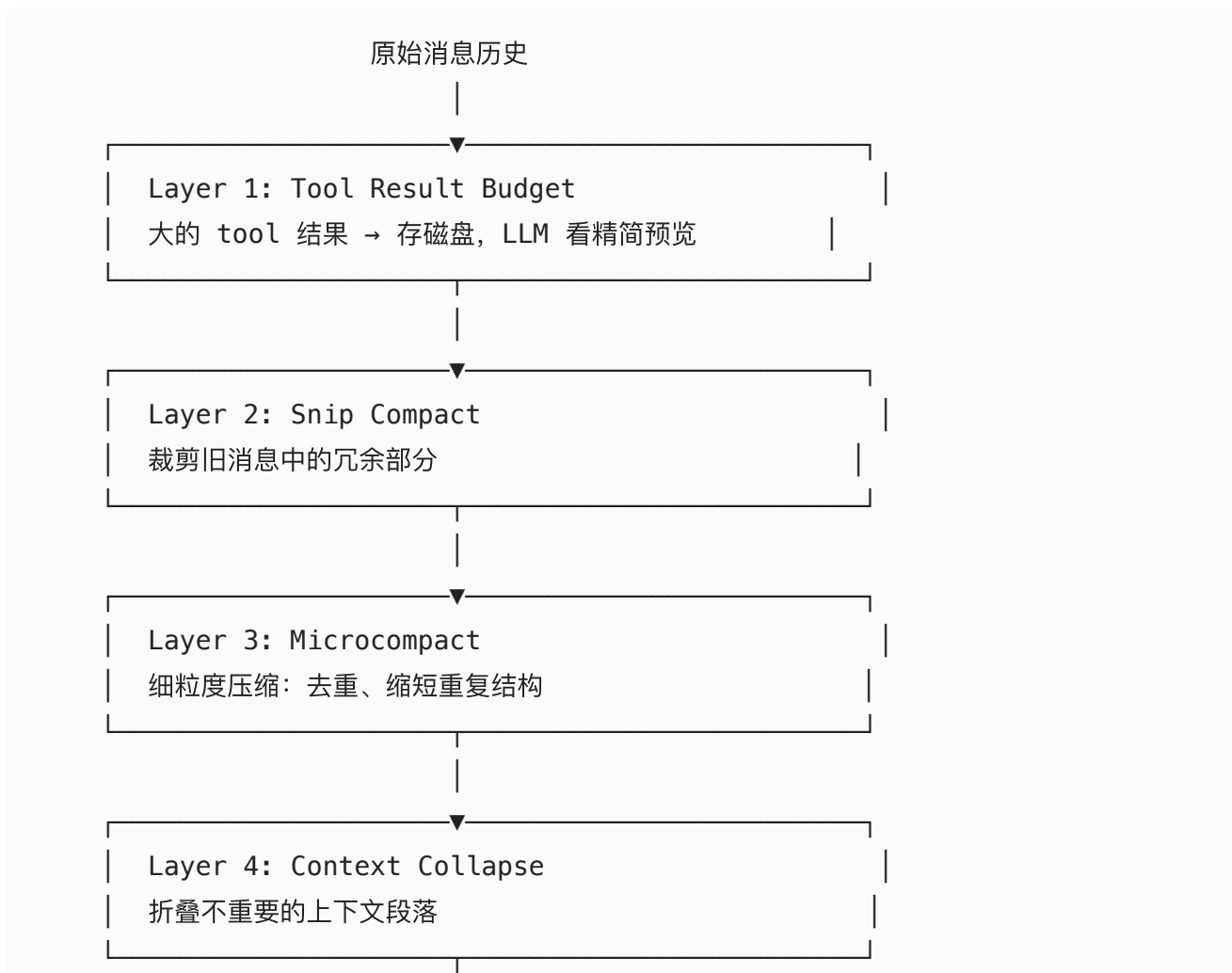
类型	内容	示例
user	用户身份、偏好、角色	"用户偏好 TypeScript"
feedback	用户的纠正和反馈	"不要自动加 docstring"
project	不可从代码推导的上下文	"截止日期是下周五"
reference	外部系统链接	"Dashboard URL: ..."

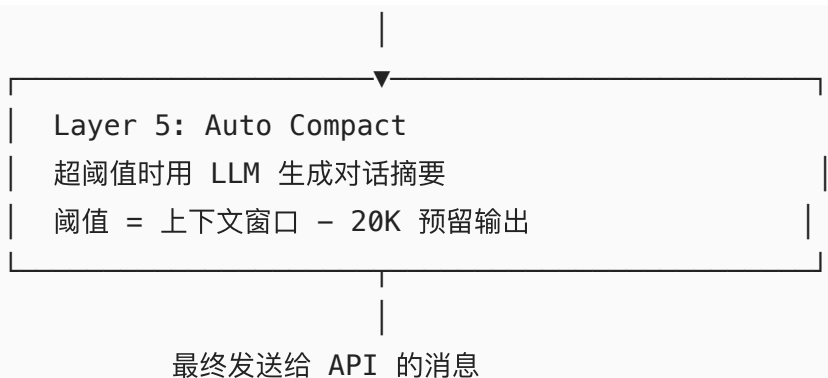
⚠ 不应存储的

可从代码推导的信息（架构、文件结构等）——这些用工具实时读取更准确。

五层压缩漏斗

Claude Code 面临的核心挑战：对话越长，消息越多，最终超出上下文窗口。解决方案是一个多层压缩管道，在 `queryLoop` 每轮开头执行：





Reactive Compact — 紧急压缩

当 API 返回 `prompt_too_long` (413) 错误时，表明 `auto compact` 估算不够准确。此时在同一轮内立即启动 `reactive compact` 压缩并重试，避免整个 `turn` 失败。

六、Multi-Agent 协作

Claude Code 支持一个主 Agent 生成多个子 Agent 并行工作，通过 Task 系统和 Message 机制协调。

Agent 生成流程

AgentTool 允许 LLM 创建子 Agent (`src/tools/AgentTool/runAgent.ts`):

1. 解析 Agent 定义
 - ├ built-in agents (Explore, Plan 等)
 - └ custom agents (`.claude/agents/*.md`)
2. `createSubagentContext()`
 - ├ Fork 父上下文的 `readFileState` (文件缓存共享)
 - ├ 独立的 `AppState` (权限上下文隔离)
 - └ 独立的 `AbortController`
3. 过滤工具集 (`resolveAgentTools`)
 - ├ Explore agent → 只有只读工具
 - ├ Plan agent → 只有只读工具
 - └ general-purpose → 完整工具集
4. 初始化 Agent 专属 MCP servers
5. 调用 `query()` ← 复用同一个核心 Agent Loop!

6. 收集结果 → 写入 sidechain transcript → 返回给父 Agent

🔗 核心洞察：递归复用

子 Agent 不是独立进程——它就是在同一进程中再次调用 `query()` 函数。Agent Loop 是递归复用的。子 Agent 自动获得所有能力：error recovery、compact、streaming tool execution.....无需维护两套代码。

Task 系统

src/Task.ts 定义了 6 种后台任务类型：

TaskType	用途	ID 前缀
local_bash	后台 shell 命令	b
local_agent	本地子 Agent	a
remote_agent	远程 Agent (via bridge)	r
in_process_teammate	进程内队友 (Team 模式)	t
local_workflow	工作流任务	w
dream	夜间记忆蒸馏	d

状态流转：pending → running → completed | failed | killed

每个 Task 有唯一 ID (类型前缀 + 8 位随机字符)，输出写入磁盘文件 `/tmp/claude/.../tasks/{id}.output`。

Team 协作模式

TeamCreate → `~/.claude/teams/{name}/config.json`
+ `~/.claude/tasks/{name}/` (共享任务板)

团队结构：

- └─ team-lead (主 Agent, 协调者)
- └─ researcher (研究子任务)
- └─ implementer (实现子任务)
- └─ tester (测试子任务)

协作机制：

- └─ TaskCreate/Update/Get/List – 共享任务看板

```
└─ SendMessage – 直接消息传递
└─ Idle 通知 – Agent 完成后自动通知 lead
```

Coordinator Mode

更高级的协作模式（通过 `CLAUDE_CODE_COORDINATOR_MODE` 环境变量启用）：

```
主线程 (Coordinator)
└─ 只保留管理工具: Agent, TaskStop, SendMessage
└─ 负责分派任务、协调进度
|
Worker Agent 1 — Bash, FileRead, FileEdit
Worker Agent 2 — Bash, FileRead, FileEdit
                  可通过 Scratchpad 共享文件
```

本质

把主 Agent 变成“只会发号施令的经理”——它自己不写代码，只负责分解任务和协调 Worker。

七、关键设计模式总结

AsyncGenerator 流水线

```
QueryEngine.submitMessage()
└─ yield* query()
    └─ yield* callModel() // LLM streaming events
└─ yield message         // tool results, progress, etc.
```

- 真正的流式处理——用户即时看到 LLM 输出
- 自然的背压控制——消费者处理不过来时生产者暂停
- Generator 的 `return()` 实现优雅取消

Feature Flags + Dead Code Elimination

```
import { feature } from 'bun:bundle'

const coordinatorModule = feature('COORDINATOR_MODE')
```



```
? require('./coordinator/coordinatorMode.js')
: null
```

Bun 的 `feature()` 在 bundle 时求值，未启用分支被 tree-shake。一份代码库同时服务内部（ant）和外部用户。

Fail-Closed 安全默认值

属性	默认值	含义
<code>isReadOnly</code>	<code>false</code>	假设会写入 → 需要权限
<code>isConcurrencySafe</code>	<code>false</code>	假设不可并发 → 串行执行
<code>isDestructive</code>	<code>false</code>	配合 <code>isReadOnly=false</code> 已够保守

Prompt Cache 感知架构

- System prompt 分为 static/dynamic 区域，static 跨用户共享缓存
- Tool 按名字确定性排序，避免顺序变化导致 cache miss
- 内置工具作为连续前缀，MCP 工具排后面
- `systemPromptSection()` 会话内缓存，`/clear` 或 `/compact` 时重算

八、深入：--resume 与 Prompt Cache 的交互

🔗 问题

会话中创建了新的 git commit, `systemContext.gitStatus` 会变——KV cache 会失效吗？`--resume` 恢复时 system prompt 会重建吗？

会话内：memoize 保护

`getSystemContext` 和 `getUserContext` 都用了 `lodash memoize()`，在整个会话生命周期内只计算一次：

```
// src/context.ts
export const getSystemContext = memoize(async () => {
  const gitStatus = await getGitStatus()
  return { ...(gitStatus && { gitStatus }) }
})
```

源码中的注释直接说明了这一点：

"This is the git status at the start of the conversation. Note that this status is a snapshot in time, and will not update during the conversation."

所以在同一个会话中，无论你创建多少个 commit，git status 始终是会话启动时的快照。只有 /clear 或 /compact 才会清除这个缓存。

--resume 时：全部缓存显式清除

--resume 在 main.tsx 中处理时，第一件事就是清除所有缓存：

```
// main.tsx line 3365-3368
if (options.resume || options.fromPr || teleport || remote !== null) {
  const { clearSessionCaches } = await
import('./commands/clear/caches.js')
  clearSessionCaches()
}
```

clearSessionCaches() 触发的清除链路：

```
clearSessionCaches()
├─ getUserContext.cache.clear()      ← CLAUDE.md 重新读取
├─ getSystemContext.cache.clear()    ← git status 重新获取
├─ getGitStatus.cache.clear()        ← git log/branch 重新拉取
└─ runPostCompactCleanup()
    └─ clearSystemPromptSections() ← 所有 systemPromptSection 缓存清除
        └─ STATE.systemPromptSectionCache.clear()
```

🔗 结论

--resume 时 system prompt 是完全重新构建的。所有 memoized 函数因缓存已清除而重新执行——git status 拿最新的，CLAUDE.md 重新读，memory 重新加载。

对 KV Cache 的精确影响

部分	resume 时变化?	Prompt Cache 影响
Static sections (Intro/System/Actions...	❌ 不变	✅ 命中
Dynamic sections (memory/env_info...	⚠️ 可能变	通常命中 (值大概率不...
systemContext (git status)	✅ 会变	❌ 这段失效

但 systemContext 通过 appendSystemContext() 放在 system prompt 最末尾：

```
// query.ts
const fullSystemPrompt = asSystemPrompt(
  appendSystemContext(systemPrompt, systemContext) // ← git status 追加到
  最后
)
```

Anthropic 的 prompt cache 是前缀匹配的：

[Static sections]	← 跨用户缓存命中 ✅
[== DYNAMIC BOUNDARY ==]	
[Dynamic sections]	← 会话内缓存命中 ✅ (大概率值不变)
[systemContext/gitStatus]	← 放最末尾，变了也只影响这一小段 ⚠️

🔗 设计洞察

这就是为什么放置顺序如此讲究——越容易变的东西越往后放，把 cache 破坏控制在最小范围。git status 是最易变的部分，刻意放在最后，确保前面数千 token 的 static + dynamic sections 的 cache 不受影响。这是整个 Prompt Cache 感知架构中最精妙的细节之一。

附录：完整数据流图

